

Week-01 HandsOn Solution

❖ Moq

1. Write Testable Code with Moq :

CustomerCommLib / MailSender.cs

```
using System.Net;
using System.Net.Mail;

namespace CustomerCommLib
{
    public interface IMailSender
    {
        bool SendMail(string toAddress, string message);
    }

    public class MailSender : IMailSender
    {
        public bool SendMail(string toAddress, string message)
        {
            MailMessage mail = new MailMessage();
            SmtpClient SmtpServer = new SmtpClient("smtp.gmail.com");

            mail.From = new MailAddress("your_email_address@gmail.com");
            mail.To.Add(toAddress);
            mail.Subject = "Test Mail";
            mail.Body = message;

            SmtpServer.Port = 587;
            SmtpServer.Credentials = new NetworkCredential("username", "password");
            SmtpServer.EnableSsl = true;

            SmtpServer.Send(mail);
            return true;
        }
    }

    public class CustomerComm
    {
        IMailSender _mailSender;

        public CustomerComm(IMailSender mailSender)
        {
            _mailSender = mailSender;
        }

        public bool SendMailToCustomer()
        {
            _mailSender.SendMail("cust123@abc.com", "Some Message");
        }
    }
}
```

```

        return true;
    }
}

```

CustomerComm.Tests/CustomerCommTests.cs

```

using Moq;
using NUnit.Framework;
using CustomerCommLib;

namespace CustomerComm.Tests
{
    [TestFixture]
    public class CustomerCommTests
    {
        private Mock<IMailSender> _mockMailSender;
        private CustomerCommLib.CustomerComm _customerComm;

        [OneTimeSetUp]
        public void Setup()
        {
            _mockMailSender = new Mock<IMailSender>();
            _mockMailSender.Setup(x => x.SendMail(It.IsAny<string>()),
It.IsAny<string>()).Returns(true);
            _customerComm = new CustomerCommLib.CustomerComm(_mockMailSender.Object);
        }

        [Test]
        public void SendMailToCustomer_ShouldReturnTrue()
        {
            bool result = _customerComm.SendMailToCustomer();
            Assert.IsTrue(result);
        }
    }
}

```

● Output:

```

Restore complete (0.4s)

CustomerCommLib succeeded
CustomerComm.Tests succeeded

NUnit Adapter 4.3.1.0: Test execution started
Running all tests in CustomerComm.Tests.dll

NUnit Adapter 4.3.1.0: Test execution complete

CustomerComm.Tests test succeeded (2.1s)

Test summary:
Total: 1
Failed: 0
Succeeded: 1
Skipped: 0
Duration: 2.1s

Build succeeded in 6.6s

```

❖ NUnit

- **NUnit Hands-On: Unit Testing Calculator Operations**

CalcLibrary/Calc.cs

```
using System;

namespace CalcLibrary
{
    interface IMathLibrary
    {
        double Addition(double a, double b);
        double Subtraction(double a, double b);
        double Multiplication(double a, double b);
        double Division(double a, double b);
    }

    public class SimpleCalculator : IMathLibrary
    {
        double result = 0;

        public double Addition(double a, double b)
        {
            result = a + b;
            return result;
        }

        public double Subtraction(double a, double b)
        {
            result = a - b;
            return result;
        }

        public double Multiplication(double a, double b)
        {
            result = a * b;
            return result;
        }

        public double Division(double a, double b)
        {
            if (b == 0)
                throw new ArgumentException("Second Parameter Can't be
Zero");
            result = a / b;
            return result;
        }

        public void AllClear()
        {
            result = 0;
        }
    }
}
```

```

        public double GetResult
        {
            get { return result; }
        }
    }
}

```

CalcLibrary.Tests/CalculatorTests.cs

```

using NUnit.Framework;
using CalcLibrary;
using System;

namespace CalcLibrary.Tests
{
    [TestFixture]
    public class CalculatorTests
    {
        private SimpleCalculator _calc;

        [SetUp]
        public void Setup()
        {
            _calc = new SimpleCalculator();
        }

        [TestCase(2, 3, 5)]
        [TestCase(-1, -2, -3)]
        public void Addition_ShouldReturnCorrectSum(double a, double b,
double expected)
        {
            var result = _calc.Addition(a, b);
            Assert.That(result, Is.EqualTo(expected));
        }

        [TestCase(5, 3, 2)]
        [TestCase(0, -2, 2)]
        public void Subtraction_ShouldReturnCorrectResult(double a, double
b, double expected)
        {
            var result = _calc.Subtraction(a, b);
            Assert.That(result, Is.EqualTo(expected));
        }

        [TestCase(4, 2, 8)]
        [TestCase(-1, 5, -5)]
        public void Multiplication_ShouldReturnCorrectResult(double a,
double b, double expected)
        {
            var result = _calc.Multiplication(a, b);
            Assert.That(result, Is.EqualTo(expected));
        }

        [TestCase(10, 2, 5)]
        public void Division_ShouldReturnCorrectResult(double a, double b,
double expected)
        {
            var result = _calc.Division(a, b);

```

```

        Assert.That(result, Is.EqualTo(expected));
    }

    [Test]
    public void Division_ByZero_ShouldThrowException()
    {
        Assert.Throws<ArgumentException>(() => _calc.Division(5, 0));
    }

    [TearDown]
    public void Cleanup()
    {
        _calc = null;
    }
}
}

```

- **Output:**

```

Restore complete (0.4s)

CalcLibrary succeeded (0.3s)
CalcLibrary.Tests succeeded (0.4s)

 NUnit Adapter 4.3.1.0: Test execution started

Running all tests in
CalcLibrary.Tests.dll

 NUnit3TestExecutor discovered 8 of 8 NUnit test cases using Current Discovery mode,
Non-Explicit run

 NUnit Adapter 4.3.1.0: Test execution complete

CalcLibrary.Tests test succeeded (1.1s)

Test summary:
total: 8
failed: 0
succeeded: 8
skipped: 0
duration: 1.1s

Build succeeded in 2.9s

```